

Agentic Automated EDA: Challenge Kick-off & Handout

The challenge

Standard interaction with a Large Language Model is a chat: you ask, it answers. The field is moving toward Agentic AI, where a system plans multi-step work, acts on an environment through tools, executes tasks iteratively, and improves its own output through self-checking.

Your group works on an experimental agentic system that automates the initial Exploratory Data Analysis (EDA) pipeline on a real scientific dataset: the system writes Python code, executes it, and tries to fix its own bugs when things go wrong. Along the way you implement and evaluate the four fundamental agentic design patterns: **reflection, tool use, planning, and multi-agent collaboration**.

Main info

1. **You build the orchestration yourself, in plain Python.** Agent frameworks (smolagents, LangChain, CrewAI, and similar) are off limits; they hide exactly the mechanics you are here to understand. Reading their documentation for ideas is allowed. Plain HTTP client libraries (openai, requests) are allowed.
2. **You use an open model running locally.** The goal is to work on your own laptops using open source small models, for example qwen3:8b (or qwen3:4b).
3. **Generated code runs in a separate process** (i.e. Docker).
4. **Every model call and every code execution leaves a trace.** Systems without traces cannot be discussed, debugged, or graded.
5. **Focus on building skills around agentic AI, not around astrophysics.** The reasoning behind selection of the current dataset is to have a challenging example that existing LLMs have not seen and memorized, which allows for better learning of envisioned concepts. It is ok to spend time to understand the problem and given data and that is normal in every kind of EDA, but you do not have to spend too much time on it, it is not the goal. What we grade is your system of working and your evaluation of the working system you are developing.

Design principle to have in mind:

When you work on designing agentic systems and when you notice that a task is too complex for one model call, ask: how would I, as a human, do this? Split the task into steps along that answer, then ask for each step: can a model call do this, can a tool do this? If neither, split again. This decomposition habit is the general method behind all four patterns.

Finally, the working method that should be adopted is as follows: **build a simple end-to-end version first, then iterate, and never change the system without measuring the change.** Look at the errors your current version actually makes, and let those observed errors decide what you build and what you measure next. Store the information about this process and use it as a basis for the presentation at the end of the challenge.

The envisioned steps: An Example

Step 0: Setup, first look, research question

Get the local environment running and load the dataset with plain pandas yourselves: shape, types, ranges, missing values, a handful of rows. Before any model touches the data, you should know several true facts about it, so that later you can catch the model when it is wrong.

Then define your **research question**: one or two sentences stating what your EDA is meant to find out about this data.

Step 1: Zero-shot baseline

Ask the local model to just do the task. Store and examine: its performance, the errors it made, the problems you observed, and the things that worked out well. Questions to answer: What can a bare model deliver on your research question, and where exactly does it fall short? This baseline is the reference point every later step is measured against.

Step 2: Reflection

Develop a reflection step: the model critiques its own output, then produces an improved version. Build reflection in three escalating levels, evaluating after each:

6. Reflection alone.
7. Reflection enriched with observed artifacts of the generation: outputs, errors, and anything else the run produced that a critic could look at. Consider what "artifacts" means for EDA, including visual ones.
8. Reflection with external feedback: facts fetched from outside the conversation (for example results of a dedicated code snippet, or a lookup) that help support or refute the output's claims.

Questions to answer: Did reflection improve the result over the baseline, and over the previous level? How well, how to measure that? What did it cost?

Step 3: Tool use

Tools are functions you describe to the model and allow it to request. Questions to think about and answer first: Which tools suit the EDA task? For example code execution is one, are there more? Is there something specific worth building for visualizations? For writing files? For fact checks? Argue for the tool set you choose.

Then build in two stages: first the tool call alone and then enhance the loop with feedback, so that a failed execution flows back and the model revises its own code, your reflection pattern from Step 2 now working inside the tool loop. Also experiment with calls that use multiple tools in one turn.

Questions to answer: How reliably does the model use each tool? When something fails, how often does the system recover without your help, and how? At what number of tools does adding more stop helping, and what takes over then?

Step 4: Planning

Planning is a design pattern where an LLM decides a sequence of steps. You ask it to plan how to solve the task and produce the list of steps, then implement those steps as a sequence of model calls. Where it makes sense, have the model produce structured output, and evaluate whether that helps.

Questions to answer: What does the model's plan look like, and does it cover what your own analysis in Step 0 says the data needs? Does execution follow the plan? Planning trades developer control for flexibility, and the system becomes more experimental and harder to steer; where did that trade-off help you, and where did it hurt? Evaluate the outcome quantitatively.

Step 5: Multi-agent collaboration

Instead of one agent doing everything, set up a system of roles pursuing the same goal. Questions to answer by design, before coding: Which roles make sense for EDA on this data? What tasks does each role own, and which tools does each role need? Compare at least two ways of wiring the agents: a fixed linear hand-off, and a manager that decides which agent to call and how to structure the work.

Questions to answer by measurement: Does the multi-agent system beat your best single-agent system on the same metrics? At what cost? Where does coordination itself fail?

Step 6: Combine, conclude

Combine the patterns based on what your measurements showed, into the best system you can argue for. Final questions, which are also the spine of your report: What is the best possible outcome you achieved, and what is the reasoning behind that configuration? How does the generated EDA report look, and how do the artifacts and traces of the system's work look? Walk through one complete trace: what the system did at each step, and how the overall behavior emerged. Where does the system still fail, and can you see it coming in the traces?

Evaluation discipline

Disciplined evaluation is one of the most important aspects of working with agentic systems, and it is graded accordingly.

- **Start from observed errors.** Build the first working version, look at what it actually gets wrong, and turn the most frequent or most damaging error into a measured quantity.
- **Two kinds of errors, two kinds of checks.** Objective errors can be verified with a code snippet against the data. Subjective qualities (the usefulness of generated text) are assessed with an LLM as a judge; the key is a detailed judging prompt that states the criteria, the scale, and what each score means, never a bare "judge this answer". Validate your judge against a few cases you scored yourselves.
- **The 2x2 matrix.** Organize your metrics along two axes, objective versus subjective and per-step versus end-to-end, and keep at least one metric in each quadrant. Which concrete metrics fill the quadrants is yours to derive.
- **Repeat same experiments multiple times.** These systems vary between runs of the same configuration; a single run proves nothing. Repeat your measurements and report the variation.
- **Track cost next to quality.** Establish procedures to track tokens and time per configuration through storing traces of executions. That information can be useful to drive decisions around next steps in real-life situations.

Traces

Develop, from day one, a habit and a mechanism for looking into your system's traces: what the system did at each step, and what it does globally across a run. Concretely: every model call and every code execution writes one structured line to a log, and every piece of generated code is kept as a file. After each experiment, read the trace before forming an opinion about what happened. Your traces and generated code are part of the submission, and reading a trace live is part of the examination. How you structure the log is your choice; that it exists and that you can navigate it is required.

The dataset

The Sun, and what a flare is

The Sun is a ball of plasma threaded by magnetic fields. Those fields store enormous energy, and where they twist and reconnect, that energy releases suddenly: a **solar flare**. A flare heats plasma to tens of millions of degrees within seconds and shines brightly in X-rays for minutes to hours. Flares are the most powerful explosions in the solar system; the largest release more energy than millions of hydrogen bombs.

At minimum, days pass without a single flare; near maximum, dozens occur per day. Your dataset spans February 2021 to February 2026 with all observed occurrences stored there.

Why anyone monitors flares

Flares, and the eruptions that often accompany them, drive **space weather**. Their radiation and particle storms disturb satellite electronics, degrade GPS accuracy, black out high-frequency radio, induce currents in power grids, and pose radiation risks to astronauts and polar flight crews. Forecasting space weather begins with knowing what the Sun is doing, and that knowledge lives in catalogs exactly like the one used in the challenge.

The two instruments in this data

GOES is a family of satellites in Earth orbit that has monitored the Sun's overall X-ray brightness continuously for decades. Flare strength is traditionally stated as a **GOES class**: a letter A, B, C, M, or X, each step ten times stronger than the last, plus a number for fine grading, so M1.5 is a medium-strong flare and X9 a huge one. Because GOES sits near Earth, it sees only the side of the Sun facing us.

STIX is the X-ray telescope aboard ESA's **Solar Orbiter**, launched in 2020. Solar Orbiter circles the Sun itself and its position changes constantly. Two consequences shape your dataset. First, STIX regularly watches the **far side of the Sun**, recording flares no Earth-bound or Earth-orbiting instrument can see; roughly half of the dataset catalog consists of such events. Second, STIX's distance to the Sun varies, and the same flare produces more counts when the spacecraft is closer. The STIX Data Center that processes this instrument's data is operated here at I4DS, FHNW. The data you are analyzing is produced at our institute.

How the measurements come about

STIX counts X-ray photons in several **energy bands**, from 4 to 84 keV. The lowest band (4-10 keV) responds to essentially every flare and serves as the general size measure; the higher bands light up only for energetic events. Counts per flare span many orders of magnitude, from a few hundred to hundreds of millions, which is typical for flare statistics: small events are vastly more common than large ones.

During very bright flares, a protective filter called the **attenuator** moves in front of the detector so it does not saturate (`att_in = True`). With the attenuator in, measured counts drop sharply, so a huge flare can show modest counts. Interpreting counts therefore always means asking: attenuator in or out, and how far was the spacecraft?

The file

https://github.com/hayesla/stix_flarelist_science/blob/main/STIX_flarelist_w_locations_20210214_20260228_version1_python.csv

This is real operational scientific data. Treat no column as guaranteed clean, consistently typed, or free of surprises; discovering, handling, and documenting what is actually in the file is part of your task, and your Step 0 look at the data is where that starts.

Data dictionary

Column	Meaning
flare_id	Catalog identifier of the flare
start.UTC, end.UTC, peak.UTC	Start, end, and peak time of the flare (UTC)
4-10 keV, 10-15 keV, 15-25 keV, 25-50 keV, 50-84 keV	X-ray counts per energy band; 4-10 keV is the general size measure, higher bands respond to strong flares
att_in	Whether the protective attenuator was inserted during the flare
hpc_x_solo, hpc_y_solo	Flare position in arcseconds from disk center, as seen from Solar Orbiter
hpc_x_earth, hpc_y_earth	The same position converted to the Earth viewpoint; empty for far-side flares
visible_from_earth	Whether the flare was on the Earth-facing side of the Sun
hgs_lon, hgs_lat	Heliographic longitude and latitude of the flare in degrees; longitude 0 is disk center as seen from Earth; empty when no position could be determined
solo_position_AU_distance	Solar Orbiter's distance from the Sun in AU (Earth-Sun distance = 1); about 0.29 to 1.02
GOES_class_time_of_flare, GOES_flux_time_of_flare	GOES class and X-ray flux (W/m ²) measured by GOES at the time of the flare
goes_estimated_mean_class, goes_estimated_mean_flux	GOES-equivalent class and flux estimated from STIX's own measurement
error_with_imaging	True if position imaging failed for this event

In this document we give some basics about the phenomena and the dataset. It is ok to learn more and experiment with the given data, but there is no need to spend too much time on solar physics since the goal remains understanding basic Agentic AI design patterns and using them to answer the selected research question.

Note: A few scientific papers are available as optional background

Data center paper: <https://www.aanda.org/articles/aa/pdf/2023/05/aa46031-23.pdf>

Correlation between STIX and GOES: <https://www.aanda.org/articles/aa/pdf/2025/02/aa52574-24.pdf>

3D image reconstruction from STIX data: <https://www.aanda.org/articles/aa/pdf/2026/06/aa58675-25.pdf>

One example of a fitting use for such information would be to use the given content as source material for a domain-expert agent in Step 5 (i.e. as a dedicated expert agent in MAS architecture).

Interesting research questions

Some of interesting research questions about flares across five years:

- **The solar heartbeat.** How did the Sun's flare activity change between February 2021 and February 2026? Concretely: how did the number of flares per month evolve, and did strong flares become a larger share of all flares throughout the time?
- **The typical flare.** What does a typical flare in this catalog look like in intensity and duration, and how far do the extremes sit from that typical case? Are the largest events rare outliers of an otherwise narrow population, or does the catalog span a continuum from tiny to enormous?
- **Two eyes on the Sun.** For flares where both instruments characterize the same event, how well do STIX's estimated strength classes agree with the classes measured from Earth? Which flares allow that comparison in the first place, and where in the strength range do the two disagree most?
- **Where flares live.** Where on the Sun do flares occur, and does that spatial pattern drift over the five years? And what did only this spacecraft see: what fraction of the catalog happened on the far side, and do those flares differ from the Earth-visible ones in strength, duration, or timing?

Note: These are directions, sharpening them into a precise, answerable research question is Step 0. Also, each of the steps should start with understanding of the observed data quality. While your research question serves as the primary driver, **what we grade is your evaluation of the working system you are developing**: identifying the optimal agent architecture to address that question, backed by empirical metrics, performance traces, and token consumption costs.